

SYSTEM DESIGN INTERVIEW

Design Pastebin

anonymous text and code paste service with expiry, syntax highlighting, and abuse controls

~ 45 minute read

© Study Guide — For personal use only

What's Inside

- 01** · Problem Statement
- 02** · Requirements
- 03** · Capacity Estimation
- 04** · High-Level Architecture
- 05** · Key Generation
- 06** · Storage Architecture
- 07** · Write Path Deep Dive
- 08** · Read Path & Cache Hierarchy
- 09** · Syntax Highlighting
- 10** · Expiry & Lifecycle
- 11** · Privacy Modes

- 12** · Abuse Prevention
- 13** · Scalability & Distribution
- 14** · Failure Modes & Recovery
- 15** · Design Trade-offs
- 16** · Interview Playbook

01

Problem Statement

Requirements and scope

Design a paste service like **Pastebin**, **GitHub Gist**, or **hastebin**. Users submit a blob of text or source code and receive a short shareable URL. The service must store the body, render it (often with syntax highlighting), serve it on demand, expire old pastes, and resist abuse.

Clarifying Questions

Question	Answer
Pastes/day?	1 million creates; 10:1 read:write → 10 million reads
Paste size?	Avg ~10 KB; cap at 10 MB; reject larger
Anonymous?	Yes; optional account for management
Expiry?	10 min / 1 h / 1 d / 1 mo / never (default 1 mo)
Privacy modes?	Public, unlisted (URL only), password-protected
Syntax highlighting?	Yes; ~50 languages, auto-detect on submit
Edits?	No; pastes are immutable. New paste = new URL
Abuse controls?	Safe Browsing scan, DMCA flow, rate limit

02

Requirements

Functional and non-functional

Functional Requirements

Requirement	Details
Create	POST /pastes { body, language?, expiry?, privacy?, password? } → short URL
Read	GET /:key → render paste with syntax highlighting
Raw	GET /:key/raw → plain text body (for curl/wget)
Expiry	Configurable TTL; lazy delete on read after expiry
Privacy	Public (listed), unlisted (URL-only), password-protected
Highlighting	Server-side render at write time; cache HTML
Abuse report	POST /report → flag for moderator review

Non-Functional Requirements

Requirement	Target
Availability	99.9% reads; 99.5% writes
Latency	<200 ms read p99 (cached); <500 ms write p99
Consistency	Read-your-writes for the creator; eventual for others
Storage	~10 GB/day raw; ~3.6 TB/yr hot tier
Throughput	~12 writes/sec; ~120 reads/sec average; 10× peak
Durability	11 nines (S3-class) for paste bodies
Key space	Base62 8-char → $62^8 \approx 218$ trillion keys

03

Capacity Estimation

Math for scale

Traffic

- Pastes created/day: **1 million**
- Read:write ratio: **10:1** (most pastes read a handful of times)
- Reads/day: $1M \times 10 = \mathbf{10\ million}$
- Write QPS avg: $1,000,000 / 86,400 \approx \mathbf{\sim 12\ writes/sec}$
- Read QPS avg: $10,000,000 / 86,400 \approx \mathbf{\sim 116\ reads/sec \approx 120/sec}$
- Peak factor $\sim 10\times \rightarrow \mathbf{120\ writes/sec, 1,200\ reads/sec}$ at peak

Storage

- Average paste size: **$\sim 10\ KB$** (10,000 bytes) of body text
- Daily body volume: $1M \times 10\ KB = \mathbf{10\ GB/day}$
- Metadata: $\sim 500\ B/row \times 1M = \mathbf{\sim 500\ MB/day}$ in Postgres
- Highlighted HTML cache: $\sim 3\times\ source \approx 30\ KB \Rightarrow \mathbf{\sim 30\ GB/day}$ if everything cached
- Annual hot tier (1-year retention of warm pastes): **$\sim 3.6\ TB/yr$**
- After 5 years with cold tiering to Glacier: $\sim 18\ TB$ total, $<4\ TB$ hot

Bandwidth

- Ingress: $12\ writes/sec \times 10\ KB \approx \mathbf{120\ KB/sec}$ (negligible)
- Egress: $120\ reads/sec \times 10\ KB \approx \mathbf{1.2\ MB/sec}$ avg; $\sim 12\ MB/sec$ at peak
- CDN absorbs the long-tail re-reads of the same paste; egress at origin $\sim 30\%$

Cache

- Hot working set: pastes accessed in the last 24 h $\approx \sim 2\text{--}3\ M$ unique keys
- At $\sim 10\ KB$ each: **$\sim 25\ GB\ Redis$** (rendered HTML kept on CDN, body kept in S3)
- Inline-in-DB shortcut: pastes $<4\ KB$ live in Postgres $\rightarrow$ no S3 round-trip

KEY

Key Numbers

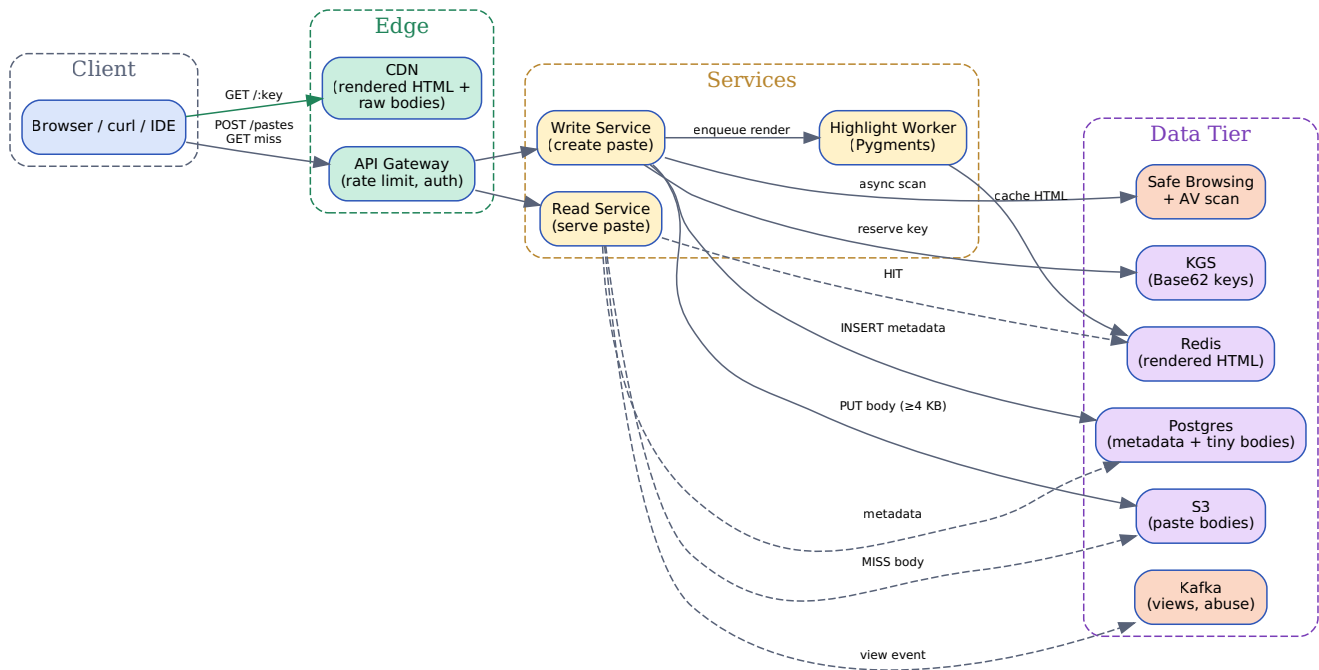
Writes: **$\sim 12/sec$ avg, $\sim 120/sec$ peak** · Reads: **$\sim 120/sec$ avg, $\sim 1.2K/sec$ peak** · Body storage: **$10\ GB/day$, $\sim 3.6\ TB/yr$ hot** · Cache: **$\sim 25\ GB\ Redis$** · Key space: **$62^8 \approx 218T$**

04

High-Level Architecture

System overview

The system splits into **Edge** (CDN + API Gateway), **Service Layer** (Write Service, Read Service, Highlight worker), and **Data Tier** (KGS, Postgres metadata, S3 paste bodies, Redis rendered-HTML cache, Kafka analytics, Safe Browsing). The *body* of a paste is large and immutable, so it lives in object storage; the *metadata* is small and queryable, so it lives in a relational store.



Top-level data flow: clients hit the CDN; misses go through the API gateway to Write/Read services; bodies sit in S3 with a Redis cache in front; analytics and abuse scanning are async.

Architecture Highlights

- **CDN:** caches both rendered HTML and raw bodies; absorbs viral pastes
- **API Gateway:** auth, rate limiting (10/hr/IP for anonymous), routing
- **Write Service:** validates, reserves a key from KGS, persists metadata + body, enqueues highlight + safety scan
- **Read Service:** serves rendered HTML from cache; falls back to S3 + on-the-fly render
- **Highlight Worker:** pulls from queue, runs Pygments, writes HTML to Redis + CDN origin
- **KGS:** pre-generated Base62 8-char keys, atomic reservation
- **Postgres:** metadata + tiny pastes inline (<4 KB) to skip S3 round-trips
- **S3:** immutable paste bodies; 11 nines durability; lifecycle to Glacier
- **Redis:** rendered HTML for hot pastes; TTL 24 h

- **Kafka:** view events, abuse signals, analytics
- **Safe Browsing / AV:** async scan; takedown if flagged

05

Key Generation

Short, unique paste IDs

Goal

Each paste gets a short, opaque, URL-safe key. Unlike a URL shortener, we are *not* hashing user input — the input is a blob of text. The key is purely an identifier. We mint it before we touch S3 so we can use it as the S3 object key and the Postgres primary key.

Comparison

Approach	Pros	Cons
UUIDv4	No coordination; trivial	26 chars in Base32; ugly URLs
MD5(body)	Dedup identical pastes for free	Long; must truncate; collision risk; leaks identical content across users
Sequential ID + Base62	Compact; ordered; small	Predictable; enumerable; needs distributed counter
KGS (pre-generated Base62)	Atomic; non-enumerable; small	Requires a service; pool refill logic

Choice: Base62 8-char from a KGS Pool

- **Alphabet:** 0–9, a–z, A–Z = 62 characters
- **Length:** 8 chars (vs URL shortener's 7) → $62^8 \approx 218$ trillion keys
- **Why 8 not 7?** Pastes are content not redirects; longer key reduces enumeration risk for unlisted pastes
- **KGS:** generates random keys, deduplicates against a Bloom filter, and hands out batches of 1,000 to each Write Service instance
- **Years to exhaust:** $218T / (1M/day \times 365) \approx \sim 600$ million years

Base62 Reference

```
import secrets, string

ALPHABET = string.digits + string.ascii_lowercase + string.ascii_uppercase
KEY_LEN = 8

def random_key():
    # 8 chars × 6 bits ≈ 48 bits of entropy
```

```
return ''.join(secrets.choice(ALPHABET) for _ in range(KEY_LEN))

# 62^8 = 218,340,105,584,896 ≈ 2.18 × 10^14 unique keys
TOTAL = 62 ** KEY_LEN
```

TIP

Why not hash the body?

Hashing the paste content would let us deduplicate identical pastes — useful for storage, but bad for privacy. Two users posting the same secret would land on the same URL. Use random KGS keys; deduplicate at the storage layer via a separate content hash if needed (S3 server-side dedup or a `body_sha256` column).

06

Storage Architecture

Metadata vs body split

Pastes have two kinds of state: small structured **metadata** (creator, expiry, language, privacy mode) and large unstructured **body** (the actual text). Putting both in Postgres works for tiny pastes but bloats the row store; putting both in S3 makes metadata queries impossibly slow. The standard answer is to split.

Split-Store Comparison

Strategy	Read latency	Storage cost	When
All in Postgres	Fastest (1 hop)	Expensive at TB scale; row size hurts vacuum	Tiny site, bodies < 1 KB
All in S3	Slow (2 hops + JSON parse)	Cheap (~\$0.023/GB-mo)	Pure archive, no queries
Postgres + S3 split	1 hop for hot, 2 for cold	Cheap bodies, fast metadata	Default for Pastebin
Postgres inline (<4 KB) + S3 (≥4 KB)	1 hop for >50% of pastes	Optimal	Production sweet-spot

Schema (Postgres)

```
CREATE TABLE pastes (
  paste_key      CHAR(8)      PRIMARY KEY,      -- Base62 KGS key
  user_id        BIGINT       NULL,              -- NULL for anonymous
  language       VARCHAR(32)  NULL,              -- 'python', 'json', ...
  privacy        SMALLINT     NOT NULL,          -- 0 public 1 unlisted 2 password
  password_hash  TEXT         NULL,              -- bcrypt; NULL unless privacy=2
  body_inline    TEXT         NULL,              -- inline if size_bytes < 4096
  body_s3_key    TEXT         NULL,              -- S3 object key otherwise
  size_bytes     INT          NOT NULL,
  body_sha256    CHAR(64)     NOT NULL,          -- for dedup / integrity
  created_at     TIMESTAMPTZ  NOT NULL DEFAULT NOW(),
  expires_at     TIMESTAMPTZ  NULL,              -- NULL = never
  retention      SMALLINT     NOT NULL,          -- enum 10m/1h/1d/1mo/never
  view_count     BIGINT       NOT NULL DEFAULT 0,
  abuse_status   SMALLINT     NOT NULL DEFAULT 0, -- 0 ok 1 pending 2 blocked
  CHECK ((body_inline IS NULL) <> (body_s3_key IS NULL))
);
```

```
CREATE INDEX idx_pastes_expires_at ON pastes (expires_at)
  WHERE expires_at IS NOT NULL;
CREATE INDEX idx_pastes_user_created ON pastes (user_id, created_at DESC)
  WHERE user_id IS NOT NULL;
CREATE INDEX idx_pastes_abuse ON pastes (abuse_status) WHERE abuse_status > 0;
```

S3 Layout

- Bucket: `pastebin-bodies-{region}`; one object per paste
- Key format: `{shard}/{paste_key}` where `shard = paste_key[:2]` — spreads partitions across S3 prefixes
- Server-side encryption (SSE-KMS); integrity checked via stored `body_sha256`
- Lifecycle: hot → S3 Standard for 30 days; → S3 IA for 11 months; → Glacier afterward
- Versioning: off (pastes are immutable; new content = new key)

Why Inline Small Pastes

- Distribution of paste sizes is heavily skewed — ~60% are < 4 KB (snippets, error logs, configs)
- Inline avoids: extra network hop, S3 GET cost (~\$0.0004/1k requests), and S3 latency tail (~30–80 ms p99)
- 4 KB threshold matches Postgres' default TOAST boundary — rows stay in the heap, no out-of-line storage
- Above 4 KB, body lives in S3 and Postgres holds only the S3 key (cheap, predictable rows)

07

Write Path Deep Dive

Creating a paste

Step-by-Step

1. Client submits: `POST /pastes { body, language?, retention, privacy, password? }`
2. API Gateway: rate limit (10/hr/IP for anonymous, 100/hr authed); reject if over
3. Validate: `size ≤ 10 MB`; language in allow-list; retention $\in \{10m, 1h, 1d, 1mo, never\}$
4. Reserve key: pop one from local KGS batch (refill from KGS service when `< 100` left)
5. Persist body: if `size < 4 KB` inline in `body_inline`; else `PUT s3://.../shard/key`
6. Persist metadata: `INSERT INTO pastes ...` with computed `expires_at`
7. Enqueue side-effects: highlight render to Kafka `render_jobs`; Safe Browsing scan to `abuse_jobs`
8. Return: `201 { url, raw_url, expires_at }` — total p99 ≤ 500 ms

KGS Batch Reservation

```
# Pseudocode: write service holds a local key buffer
BATCH_SIZE = 1000
REFILL_AT = 100
local_keys = collections.deque()

def get_key():
    if len(local_keys) <= REFILL_AT:
        threading.Thread(target=refill, daemon=True).start()
    if not local_keys:
        # cold start fallback
        local_keys.extend(kgs.reserve(BATCH_SIZE))
    return local_keys.popleft()

def refill():
    new = kgs.reserve(BATCH_SIZE)
    local_keys.extend(new)

# KGS guarantees: each key handed out exactly once across all instances.
# UNIQUE constraint on pastes.paste_key is the safety net.
```

Highlight on Write vs on Read

Strategy	Pros	Cons
Highlight on write (server-side)	Read path is $O(\text{memcpy})$; CDN can cache HTML; consistent rendering	Wastes work for never-read pastes; render time adds to write latency unless async
Highlight on read (server-side, cached)	Don't pay for never-read pastes	First reader is slow; thundering herd on viral pastes
Highlight on read (client-side, e.g. highlight.js)	Server stays trivial	JS bundle ~100 KB; doesn't help raw / curl / no-JS clients; CPU on client

TIP

Decision: server-side highlight, async at write

Render with **Pygments** (or shiki for JS) inside the highlight worker, triggered by the Kafka `render_jobs` topic. The Read Service first checks Redis for rendered HTML; on miss it falls back to rendering inline (and warms the cache) so the first reader of a brand-new paste never waits for the worker. We pay render cost once per paste, the result is cacheable on the CDN, and curl/IDE clients still get plain text via `/raw`.

Error Handling

- **S3 PUT fails:** retry with exponential backoff; if persistent, return 503; metadata row is never inserted (S3 first, DB second)
- **Postgres insert fails after S3 PUT:** orphaned object; nightly reaper deletes S3 objects whose key has no matching row
- **KGS unavailable:** fall back to local random + UNIQUE constraint retry (max 3 retries before 503)
- **Body too large:** 413 Payload Too Large; suggest user split or use a different service
- **Safe Browsing async fails:** paste created but flagged `abuse_status=1` pending; reviewed within 1 h

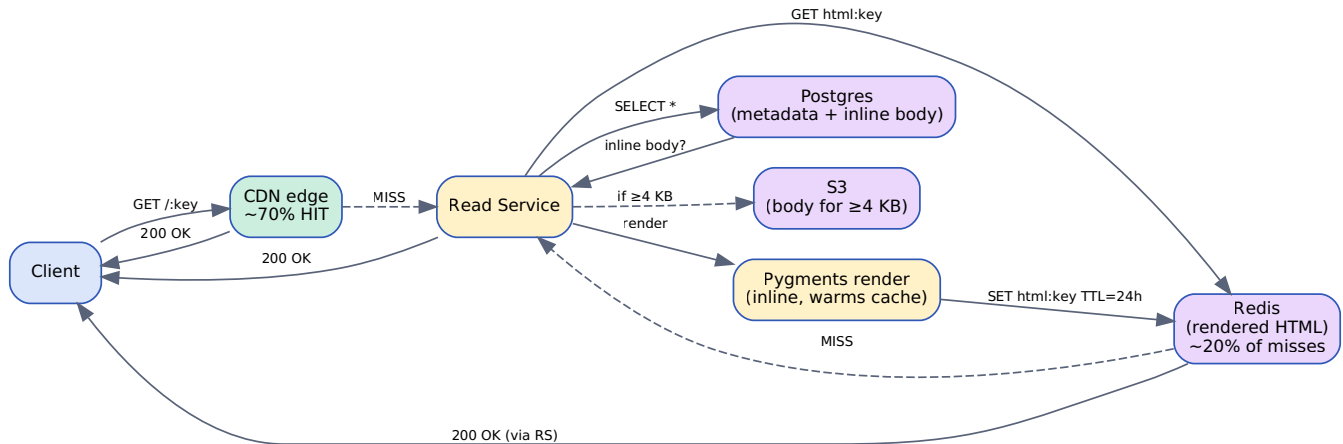
08

Read Path & Cache Hierarchy

Serving a paste fast

Cache Hierarchy

Reads traverse a four-level hierarchy. Each level gets faster but smaller. The expected hit-rate distribution for a viral paste roughly looks like: CDN ~70%, Redis ~20%, Postgres metadata + S3 body ~10%.



Read path: each level falls through to the next. CDN serves rendered HTML directly; Redis serves rendered HTML to origin; Postgres provides metadata + (sometimes) inline body; S3 holds the rest.

Step-by-Step

1. Client requests GET /abc12345
2. CDN edge: serves cached HTML if present (TTL 1 h, longer for popular)
3. Miss CDN → Read Service
4. Redis lookup: GET html:abc12345 — if hit, return immediately and let CDN cache
5. Miss Redis: SELECT * FROM pastes WHERE paste_key = 'abc12345'
6. Check expiry: if expires_at < now() → 404 + lazy delete
7. Check privacy: if password-protected, demand auth (401) or session token check
8. Body fetch: use body_inline if present; else GET s3://.../shard/key
9. Render: Pygments by language; produce HTML; SET html:key with TTL = 24 h
10. Async: publish view_event to Kafka; increment view_count via async aggregator
11. Return 200 with HTML; CDN caches for 1 h based on Cache-Control headers

Cache TTLs

Layer	TTL	Reason
CDN edge	1 h (10 min for unlisted)	Pastes are immutable but expiry is dynamic; short TTL bounds staleness
Redis HTML	24 h	Re-render on miss is cheap; longer wastes memory on cold pastes
Postgres row cache (PG buffers)	managed by PG	Hot rows naturally pinned in shared_buffers
S3 body	n/a (immutable)	Object never changes; only deleted on expiry

WARNING

Cache Bypass for Password-Protected Pastes

Never let a CDN cache the rendered HTML of a password-protected paste — a second visitor would see the body without the password. Mark such responses `Cache-Control: private, no-store`. The auth handshake happens at the Read Service; CDN caches only the auth-prompt page.

09

Syntax Highlighting

Pygments at write, cached forever

Pipeline

1. Write Service publishes `render_jobs { paste_key, language }` to Kafka
2. Highlight Worker consumes the topic; pulls body (inline or S3); runs Pygments
3. Output is HTML with inline span classes (e.g. `<span class="k">def</span>`)
4. Worker writes to Redis: `SET html:<key> <html> EX 86400`
5. First reader either gets cached HTML (Redis) or triggers an inline render fallback (and warms the cache)

Highlight + Cache-fill (Read Service Fallback)

```
from pygments import highlight
from pygments.lexers import get_lexer_by_name, guess_lexer
from pygments.formatters import HtmlFormatter

FORMATTER = HtmlFormatter(cssclass='paste', linenos='inline', wrapcode=True)

def render_and_cache(paste_key, body, language):
    try:
        lexer = get_lexer_by_name(language) if language else guess_lexer(body)
    except Exception:
        lexer = get_lexer_by_name('text') # fallback: no highlighting
    html_out = highlight(body, lexer, FORMATTER)

    # SETNX so we don't clobber a worker that won the race
    pipe = redis.pipeline()
    pipe.set(f'html:{paste_key}', html_out, ex=86400, nx=True)
    pipe.execute()
    return html_out
```

Language Auto-Detect

- Client may submit `language`; if not, server runs `guess_lexer(body)`
- Pygments guesses by token frequency / regex hints; ~80% accurate on samples > 200 chars
- On low-confidence guesses, fall back to `text` lexer (no highlighting; still escaped)
- Persist the chosen language in Postgres so re-renders are deterministic

Why Server-Side

- **curl/wget/IDE clients** get plain text via / raw; no JS execution
- **CDN-cacheable**: rendered HTML is a static blob, sized in tens of KB
- **Consistent**: every viewer sees the same colorization, regardless of browser
- **SEO/preview**: link unfurlers (Slack, Twitter) get fully rendered code in the OG image preview

INSIGHT

The Cost of a Render

Pygments highlights ~10 MB/sec/core for typical source. A 10 KB paste takes ~1 ms of CPU. At 12 writes/sec we use < 2% of a single core for rendering. The bottleneck is not CPU — it is the Redis write and HTML-versus-source size ratio (HTML is ~2–4× the source). Budget cache size accordingly.

10

Expiry & Lifecycle

TTL, lazy delete, cold tiering

Retention Options

Option	expires_at	Use Case
10 minutes	now() + 10 min	One-time secrets, paste-and-forget
1 hour	now() + 1 h	Quick code review snippet
1 day	now() + 24 h	Default for ephemeral chats
1 month	now() + 30 d	Default for casual sharing
Never	NULL	Public archive, blog post

Lazy Delete on Read

1. On every read, check `expires_at < now()`
2. If expired: return 410 Gone; mark row for cleanup; delete cached HTML from Redis
3. Don't synchronously delete the S3 object — that costs a round-trip on the read path
4. A nightly reaper job picks up rows flagged for cleanup and deletes their S3 objects in batch

Background Reaper

```
-- Pages expired pastes in chunks; runs every 30 min off-peak
WITH expired AS (
  SELECT paste_key, body_s3_key
  FROM pastes
  WHERE expires_at IS NOT NULL
    AND expires_at < NOW() - INTERVAL '5 minutes'
  ORDER BY expires_at
  LIMIT 10000
  FOR UPDATE SKIP LOCKED
)
DELETE FROM pastes
USING expired
WHERE pastes.paste_key = expired.paste_key
RETURNING pastes.paste_key, pastes.body_s3_key;
-- caller batches RETURNING into S3 DeleteObjects (1000 keys per call)
```

S3 Lifecycle Policy

- **0–30 days:** S3 Standard — frequent reads expected
- **30–365 days:** S3 Standard-IA (~50% cheaper, retrieval fee)
- **365+ days:** S3 Glacier Instant Retrieval (~\$0.004/GB-mo)
- Lifecycle moves are free if > 128 KB and ≥ 30 days; we batch-tag at write time
- Pastes with expiry ≤ 30 d never tier — they die in Standard before policy fires

Why Lazy + Reaper, not Cron-only

- Lazy on read: zero work for pastes that nobody re-reads after expiry
- Reaper: catches the long tail (pastes that expired and were never re-read) — frees S3 storage
- Combined approach amortizes deletion cost; no thundering-herd at midnight

11

Privacy Modes

Public, unlisted, password-protected

Three Modes

Mode	Discoverability	Auth	Indexed?
Public	Listed in /trending and search	None	Yes
Unlisted	Only via direct URL	None	No (robots noindex)
Password-protected	Only via direct URL	Password challenge	No

Implementation

- **Public:** `privacy=0`; surfaced in trending feed; Cache-Control public
- **Unlisted:** `privacy=1`; not in trending; X-Robots-Tag: `noindex, nofollow`; key length (8 chars) makes brute-force enumeration impractical ($62^8 \approx 218T$)
- **Password:** `privacy=2`; bcrypt-hashed password in `password_hash`; client supplies password → server verifies → issues short-lived JWT cookie scoped to that `paste_key`
- **Cache rule:** Cache-Control: `private, no-store` for password mode; CDN must not cache decoded HTML

Password Verification Flow

1. Client GETs `/abc12345`
2. Read Service sees `privacy=2`; renders password prompt page (cacheable, no body)
3. Client POSTs `/abc12345/unlock { password }`
4. Server bcrypt-verifies; if OK, issues HttpOnly JWT cookie `paste_token=...` scoped to `paste_key`, TTL 1 h
5. Subsequent GETs include the cookie; Read Service verifies signature and serves the paste

WARNING

Unlisted is not private

Unlisted means "not advertised" — the URL is still secret-by-obscurity. Anyone the link is shared with can re-share it. Tell users this explicitly. For real secrecy, recommend password mode or end-to-end client-side encryption (the PrivateBin model: encrypt body in browser, key lives in URL fragment, server stores only ciphertext).

12

Abuse Prevention

Malware, phishing, copyright, rate limits

Threat Model

Threat	How	Mitigation
Malware/phishing payload	Paste contains exploit kit, credentials harvest, etc.	Async Safe Browsing scan; flag → block
DMCA / copyright violation	Pirated source code, leaked book chapter	Takedown form; manual review; legal hold
Spam flood	Bot creates thousands of SEO-bait pastes	Rate limit 10/hr/IP anonymous, 100/hr authed; CAPTCHA on suspicious patterns
Doxxing / PII leak	Personal info, credit cards in paste body	Regex pre-scan (PAN, SSN); user-report flow; right-to-delete
Resource exhaustion	Single client uploads many 10 MB pastes	Per-IP daily byte cap (e.g. 100 MB/day anon); 413 above 10 MB single-paste
Enumeration	Attacker scans short keys to find unlisted pastes	8-char keys ($62^8 \approx 218T$); CDN/WAF burst limit; honeypot keys

Safe Browsing Pipeline

1. Write Service publishes `abuse_jobs { paste_key }` to Kafka
2. Abuse Worker fetches body; extracts URLs (regex); calls Google Safe Browsing API in batches of 500
3. If any URL is UNSAFE: `UPDATE pastes SET abuse_status = 2 WHERE paste_key = ?`; evict cache; return 451 on read
4. Otherwise: `UPDATE pastes SET abuse_status = 0` (cleared)
5. Pending state during scan: paste is viewable but tagged "under review" in metadata response

DMCA Takedown Flow

1. Rights-holder submits `POST /report` with paste URL, evidence, sworn statement
2. Trust & Safety queue: SLA 24 h for review
3. Reviewer marks `paste abuse_status = 2`; original body archived under legal hold (90 d)
4. Counter-notice flow: original poster (if authed) can dispute; auto-restore after 14 d if no court order
5. Public transparency report: monthly aggregate counts (no paste content)

Rate Limiting

```
# Sliding-window rate limit at API gateway (Redis)
ANON_LIMIT  = 10  # writes per hour per IP
AUTH_LIMIT  = 100
WINDOW_SEC  = 3600

def allow_write(ip, user_id):
    key = f'wl:{user_id or ip}'
    limit = AUTH_LIMIT if user_id else ANON_LIMIT
    now = int(time.time())
    pipe = redis.pipeline()
    pipe.zremrangebyscore(key, 0, now - WINDOW_SEC) # drop old
    pipe.zcard(key) # count in window
    pipe.zadd(key, {f'{now}-{uuid()}': now}) # add this hit
    pipe.expire(key, WINDOW_SEC)
    _, count, _, _ = pipe.execute()
    return count < limit
```

13

Scalability & Distribution

Growing past the prototype

Horizontal Scaling

- **Write Service:** stateless behind ALB; scale on CPU + queue depth
- **Read Service:** stateless; scale independently on QPS
- **Highlight Worker:** Kafka consumer group; partition by paste_key for ordering; scale on lag
- **KGS:** 2–3 instances; each reserves a non-overlapping numeric range from the master pool

Postgres Sharding

- **Shard key:** $\text{hash}(\text{paste_key}) \% N$; start $N=16$, scale to 64+
- **Per-shard:** primary + 2 replicas; read replicas absorb spikes
- **Tooling:** Citus, Vitess, or Postgres logical replication; failover via Patroni
- **Why shard early:** retention=never pastes accumulate forever; metadata grows unbounded

Redis Cluster

- **Topology:** 6 nodes (3 primary, 3 replica); ~25 GB working set
- **Sharding:** hash slot of paste_key; consistent hashing handles node moves
- **Eviction:** allkeys-lru — rendered HTML is regenerable
- **Persistence:** RDB snapshots only; no AOF (cache, not source of truth)

Global Distribution

- **Primary region:** writes pin to one region (e.g. us-east-1) for strong consistency
- **Read replicas:** Postgres async replicas in eu-west-1, ap-southeast-1 (~1–2 s lag)
- **S3:** Cross-Region Replication for paste bodies; reads served from nearest region
- **CDN:** ~200 PoPs globally; absorbs > 70% of read traffic
- **Acceptable lag:** a fresh paste may not be readable in eu-west-1 for ~2 s — fine for the use case

INSIGHT

Read replica vs CDN

For a paste service, the CDN is the dominant scale lever — most reads hit the same 0.1% of pastes (the viral ones). Read replicas help with the long tail of *different* pastes being read once each. Spend cache budget on the CDN first; replicas second.

14

Failure Modes & Recovery

What can go wrong

Failure	Impact	Detection	Mitigation
Redis down	Every read renders inline; ~5x CPU on Read Service	Redis health check; latency spike	Failover to replica; degrade to on-the-fly render only
S3 region outage	Reads of ≥ 4 KB pastes fail in that region	PUT/GET 5xx	Serve from replicated bucket in secondary region; new writes still work via inline if < 4 KB
Postgres primary down	All writes fail; reads on that shard fail	Connection timeout	Patroni promotes replica (~30 s); writes buffer on client and retry
KGS exhausted/down	Cannot create new pastes	KGS error metric	Standby instance; fall back to local random + UNIQUE retry
CDN edge down	Slow reads from that region	Provider alert; latency	Reroute to nearest edge; origin can absorb 100% briefly
Highlight worker lag	New pastes show plain (escaped) text for ~minutes	Kafka consumer lag metric	Inline render fallback in Read Service papers over the lag
Safe Browsing API down	New pastes stay in pending review	HTTP error rate	Allow viewing with banner; re-scan when API recovers
Spam flood / DDoS	Write QPS spikes; Postgres connections exhausted	Anomaly detector	WAF rules; raise CAPTCHA threshold; throttle anonymous to 1/hr

Disaster Recovery Targets

- **RTO:** < 15 min for read service; < 60 min for write service
- **RPO:** < 5 min for paste bodies (S3 CRR); < 1 min for metadata (synchronous WAL replication)
- **Backups:** Postgres base backup nightly + WAL archive; S3 versioning for accidental delete (30-day window)
- **DR drill:** quarterly region failover; chaos test on Redis loss

WARNING

Graceful Degradation

Reads are the core use case — protect them. If Redis is down, render inline. If highlighting fails, serve plain text. If S3 is slow, serve / raw from cache and hide the rendered view. Writes can fail loudly with 503; readers must always see *something* for a known paste.

15

Design Trade-offs

Decisions and rationale

Decision	Choice	Trade-off
Body storage	Postgres inline if <4 KB else S3	1 hop for ~60% of pastes; 2 hops + S3 cost for the rest. Simpler designs (all-S3 or all-PG) hurt either tail latency or storage cost.
Key generation	KGS pre-generated Base62 8-char	Extra service vs near-zero collisions and non-enumerable keys. Random + retry is simpler but flaky under contention.
Highlight timing	Server-side, async at write	Render once, cache forever; first read may inline-render. Client-side (highlight.js) is simpler but breaks curl and adds 100 KB JS.
Highlighter	Pygments	Mature, ~500 lexers, ~10 MB/sec/core. Tree-sitter is faster and more accurate but harder to deploy in Python services.
Privacy default	Unlisted (not public)	Friendlier default for users sharing logs/configs. Public-by-default would feed a trending feed but invite accidental leaks.
Expiry default	1 month	Most pastes are ephemeral; "never" is opt-in. Saves storage. Loses the archive use case unless user explicitly chooses never.
Edits	Immutable; new paste = new URL	Caching becomes trivial; CDN can cache forever. Users who want edits use a Gist instead.
Database	Postgres sharded	Strong consistency, real schema, JSONB for flexibility. NoSQL would scale further but lose the abuse-status query patterns.
Anonymous default	Anonymous OK; auth for management	Lowers friction (the use case). Costs us abuse vectors; rate limits and Safe Browsing reclaim them.
Cache layer	CDN + Redis (rendered HTML)	Two layers feel redundant but solve different problems: CDN handles geo + viral; Redis handles cold-but-not-frozen.

16

Interview Playbook

How to present this in 45 minutes

Pastebin is the canonical "URL shortener for content" interview question. The trap is treating it as a URL shortener clone — you need to address *body storage*, *rendering*, *privacy*, and *abuse* in ways the shortener doesn't.

45-Minute Outline

1. **Intro (3 min):** clarify scope — anonymous OK, immutable, ~1M pastes/day, 10:1 read:write, 10 KB avg, expiry options, syntax highlighting needed
2. **Capacity (5 min):** 12 writes/sec avg, 120 reads/sec avg, 10× peak, 10 GB/day body, 3.6 TB/yr hot, ~25 GB Redis cache
3. **Architecture (5 min):** CDN, API gateway, Write/Read services, KGS, Postgres metadata, S3 body, Redis HTML cache, Kafka, Safe Browsing — draw the diagram
4. **Storage split (8 min):** why metadata in Postgres, body in S3, <4 KB inline; show the schema; explain TOAST boundary
5. **Write path (5 min):** KGS reservation; S3 PUT before Postgres INSERT to avoid orphaned rows; async render + scan
6. **Read path (6 min):** CDN → Redis → Postgres → S3 fallback; render inline if cache miss; show the diagram
7. **Highlighting (3 min):** server-side Pygments, async, cached HTML; defend over client-side
8. **Privacy + abuse (5 min):** public/unlisted/password modes; rate limits; Safe Browsing; DMCA flow
9. **Expiry (2 min):** lazy on read + nightly reaper; S3 lifecycle to Glacier
10. **Failures (3 min):** top 3 — Redis loss, S3 region outage, KGS exhausted

Common Follow-ups & Answers

- **Q: Why split body and metadata?** A: Bodies are 10 KB on average and unstructured; rows in Postgres get fat. S3 is purpose-built for blobs at \$0.023/GB-mo, vs Postgres' ~\$0.10+/GB-mo with operational overhead. Metadata stays in Postgres because we need to query expiry, privacy, and abuse_status.
- **Q: Why server-side highlighting?** A: / raw users (curl, IDEs) get plain text; HTML users get a CDN-cacheable rendered blob. Client-side breaks no-JS clients and bloats the page by ~100 KB.
- **Q: How do you prevent unlisted pastes from being scraped?** A: 8-char Base62 → 218T keyspace; 1M valid pastes means 1 in 218 million guesses hits. Combined with WAF burst limits and honeypot keys, enumeration is impractical.
- **Q: What if a paste body never makes it to S3 but the row was inserted?** A: We PUT to S3 first, then INSERT into Postgres. If S3 fails, no row exists. If Postgres fails after S3 succeeds, we have an orphan S3 object — cleaned up nightly by a reaper that diffs S3 keys against Postgres rows.

- **Q: Password-protected paste — how do you stop the CDN from caching it?** A: Cache-Control: private, no-store on the rendered response. The CDN caches only the password prompt page (which has no body).
- **Q: How do you handle a viral paste — say a leaked source code that gets 1M views in an hour?** A: CDN absorbs nearly all of it (it's an immutable rendered HTML page). Origin sees only the cache-fill miss in each PoP — at most a few hundred origin hits regardless of view count.
- **Q: Why immutable?** A: Caching becomes trivial — CDN and Redis can cache effectively forever (until expiry). Editable pastes need versioning, cache invalidation, and a different UX. If you want edits, you want Gist, not Pastebin.

Key Talking Points

- “1 M pastes/day, 10:1 read:write, 10 KB avg → 10 GB/day, 3.6 TB/year hot” — capacity in one breath
- “Postgres for metadata, S3 for body, inline if < 4 KB” — the storage split is the headline answer
- “KGS Base62 8-char, $62^8 \approx 218T$ keys” — answers both uniqueness and enumeration concerns
- “Server-side Pygments, async on write, cached HTML” — explains why curl and the CDN both work
- “Lazy expiry on read + nightly reaper” — covers correctness and cost
- “Public, unlisted, password — three modes, three Cache-Control policies”
- “Safe Browsing async + DMCA queue” — abuse story without slowing the write path

KEY

Memorize These Numbers

1M pastes/day · 10:1 read:write · ~10 KB avg paste · 12 writes/sec avg, 120 reads/sec avg (10× peak) · 10 GB/day body, 3.6 TB/yr hot · $62^8 \approx 218$ trillion keys · <4 KB inline in Postgres else S3 · Redis ~25 GB rendered HTML cache · Anon rate limit 10/hr/IP.